HARI 2

Resiliency dan gRPC untuk Komunikasi Antarservice

Timeout, *retry*, dan *circuit breaker* dengan failsafe-go; kontrak protobuf dan gRPC untuk panggilan internal; saga dan *idempotency* sebagai fondasi Hari 3

DISIAPKAN UNTUK

Peserta In-house Training

Endy Muhardin

PT ArtiVisi Intermedia · Oktober 2026

HARI 2

Agenda hari ini

Waktu	Sesi	Format
08:00 – 08:30	Rekap Hari 1 dan penyamaan posisi dari *checkpoint* `lab-02` bagi peserta yang tertinggal	Diskusi
08:30 – 09:45	Peta *concern* Hari 2. Kegagalan komunikasi sinkron: *timeout*, *retry* dengan *backoff*, *circuit breaker*, dan alasan memilih failsafe-go	Presentasi
09:45 – 10:45	**Lab 3**: failsafe-go pada panggilan transfer ke account, simulasi service lambat dan mati	*Hands-on*
11:00 – 12:00	gRPC untuk komunikasi internal (1): perbandingan REST dan gRPC, kontrak protobuf, *code generation*	Presentasi + demo
12:00 – 13:00	Istirahat makan siang	—
13:00 – 13:30	gRPC (2): *unary call*, *error handling*, *interceptor*	Presentasi + demo
13:30 – 15:00	**Lab 4**: memindahkan panggilan transfer ke account dari REST ke gRPC	*Hands-on*
15:15 – 16:45	Konsistensi data terdistribusi: alasan 2PC dihindari, saga dengan kompensasi di studi kasus (persiapan Lab 5), *idempotency key*	Presentasi + demo
16:45 – 17:00	Ringkasan Hari 2 pada peta *concern*	Diskusi

CATATAN

Hari 2 dimulai dari *checkpoint* `lab-02`. Peserta yang belum menyelesaikan Lab 2 menjalankan `git checkout lab-02` sebelum Lab 3.

PETA CONCERN MICROSERVICES

Concern yang dibahas di Hari 2

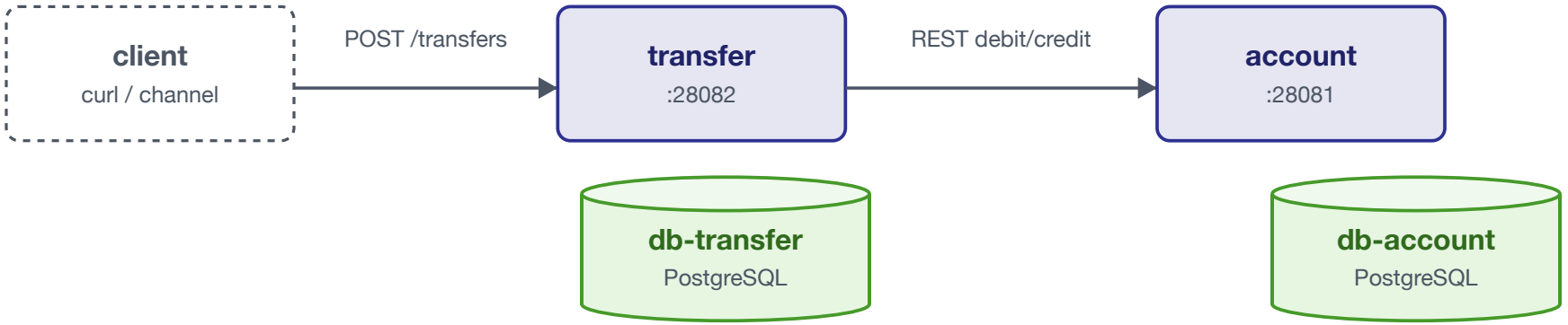
#	*Concern*	Pertanyaan yang dijawab	Hari dan lab	Di kode (*library*)	Di platform atau infrastruktur
1	*Decomposition* dan kepemilikan data	*Service boundary* dan pemilik data	H1, Lab 1–2	—	*Database per service*
2	**Komunikasi sinkron dan asinkron**	REST, gRPC, *messaging*	H1–H3, Lab 2, 4, 5	`net/http`, grpc-go, kafka-go	Kafka
3	***Resiliency***	Tetap berfungsi saat dependensi lambat atau mati	H2, Lab 3–4	failsafe-go	*Probe*, *restart*, *rolling update* (H4)
4	**Konsistensi data**	Perubahan data lintas database	H2–H3, Lab 5–6	Saga, *outbox*, *idempotency*	—
5	*Security*	Identitas pengguna (SSO) dan service, serta hak akses	H3, Lab 7	JWT *middleware* dan *interceptor*	*Authorization server* atau IdP (Hydra di lab, Keycloak), API *gateway*
6	*Observability*	Kondisi tiap service dan integrasi antarservice	H3–H4, Lab 8	`slog`, Prometheus *client*, OTel SDK	Grafana, Loki, Tempo, Prometheus
7	*Configuration* (termasuk *centralized config* dan *secret*)	Nilai yang berbeda per *environment*	H1, Lab 1 (*env var*); H4, Lab 9 (ConfigMap, Secret)	`LoadConfig`	ConfigMap, Secret, *secret manager*
8	*Service discovery* dan *load balancing*	Alamat service tujuan	H4, Lab 9	— (alamat dari konfigurasi)	Kubernetes Service (DNS), Ingress
9	*Packaging* dan *deployment*	Cara service dijalankan dan diperbarui	H4, Lab 9	Dockerfile	Kubernetes Deployment, *probe*, *rolling update*
10	*Automation* (CI/CD)	Dari *commit* ke *production*	H4, Lab 10	Target `Makefile`	*Pipeline* (git server dan runner CI, Jenkins)

CATATAN

Baris yang disorot menjadi fokus hari ini: gRPC untuk komunikasi internal, *resiliency* di sisi kode, dan desain saga untuk konsistensi data. Semua yang dibangun di Hari 2 berada di kode.

TITIK AWAL

Kondisi studi kasus di akhir Lab 2



Sudah berjalan

- Dua service dengan database masing-masing
- Transfer memanggil account melalui REST untuk debit lalu credit
- Account sudah dilengkapi *fault injection* di `/admin/fault`

Celah yang ditutup hari ini dan besok

- HTTP client di transfer belum memiliki *timeout*
- Debit tidak dikembalikan bila credit sesudahnya gagal (Hari 3)
- Kontrak REST internal hanya diperiksa saat program berjalan

01

Kegagalan Komunikasi Sinkron

Timeout, *retry* dengan *backoff*, dan *circuit breaker*

KONSEP

Panggilan antarservice bisa gagal kapan saja

- Di monolith, pemanggilan fungsi tidak pernah hilang di tengah jalan. Di microservices, panggilan ke service lain melewati jaringan, sehingga bisa gagal sebelum sampai atau saat jawabannya kembali.
- Service tujuan bisa mati, lambat, kehabisan koneksi database, atau sedang di-*deploy* ulang. Gangguan seperti ini terjadi hampir setiap hari, tanpa harus menunggu insiden besar.
- Sebagian sistem gagal sementara bagian lain tetap berjalan. Kondisi ini disebut *partial failure*.
- Pemanggil tidak bisa tahu apakah service tujuan belum memproses *request*, atau sudah memproses tetapi *response*-nya hilang di jalan.
- Karena itu, setiap panggilan ke service lain butuh aturan yang jelas: berapa lama menunggu, kapan mengulang, dan kapan berhenti mencoba.

> **TIPS**
>
> Saat meninjau kode yang memanggil service lain, ajukan dulu pertanyaan ini: apa yang terjadi bila service tujuan tidak menjawab selama satu menit?

KONSEP

Kegagalan antarservice sengaja diuji dengan *chaos engineering*

Partial failure begitu sering terjadi sehingga muncul disiplin tersendiri untuk mengujinya. *Chaos engineering* menguji sistem dengan sengaja menimbulkan gangguan, agar tim yakin sistem tetap berfungsi saat gangguan sungguhan terjadi di *production*.

Prinsip	Artinya
Build a hypothesis around steady state behavior	Tentukan dulu ukuran kondisi normal, misalnya jumlah transfer berhasil per menit
Vary real-world events	Tiru gangguan sungguhan: service mati, lambat, atau jaringan putus
Run experiments in production	Hasil yang paling jujur didapat dari sistem yang melayani *traffic* sungguhan
Automate experiments to run continuously	Eksperimen dijalankan rutin, tidak hanya sekali
Minimize blast radius	Mulai dari cakupan kecil agar dampaknya terbatas

Sudah menjadi industri

- Netflix memperkenalkan Chaos Monkey, yang mematikan *instance* secara acak di *production*. Chaos Monkey tidak dirilis lagi sejak Januari 2025.
- *Open source* untuk Kubernetes: Chaos Mesh dan LitmusChaos, keduanya proyek CNCF tahap *incubating* sejak 2022.
- Layanan berbayar: Gremlin, AWS Fault Injection Service, dan Azure Chaos Studio.

Lab 3 menerapkan versi kecilnya: *fault injection* di account service untuk meniru service yang lambat dan yang gagal.

Referensi: `principlesofchaos.org`.

KONSEP

Service yang lambat lebih berbahaya dibanding service yang mati

Service tujuan mati

- Koneksi langsung ditolak (*connection refused*).
- Pemanggil mendapat *error* dalam hitungan milidetik.
- *Goroutine* dan koneksi langsung dilepas.
- Masalahnya cepat terlihat karena *error rate* naik.

Service tujuan lambat

- Koneksi diterima, tetapi jawabannya tidak kunjung datang.
- Tanpa *timeout*, pemanggil menunggu tanpa batas.
- Selama menunggu, *goroutine*, koneksi, dan memori tetap tertahan.
- Masalahnya sulit terlihat: semua *request* ikut lambat, tetapi tidak ada *error*.

PERHATIAN

`http.Client{}` tanpa *field* `Timeout` dan tanpa *deadline* di `context` akan menunggu selamanya. Transfer service di akhir Lab 2 masih seperti ini.

KONSEP

Cascading failure (kegagalan berantai)



1. Database account melambat, sehingga setiap *request* ke account menunggu lebih lama.
2. Transfer terus mengirim *request* baru sementara *request* lama belum selesai. *Goroutine* dan koneksi HTTP di transfer bertambah tanpa batas.
3. Transfer kehabisan memori atau koneksi, lalu ikut lambat bagi channel, termasuk untuk *endpoint* yang tidak menyentuh account.
4. Channel mengulang *request* yang lambat, sehingga beban ke seluruh rantai bertambah.

KONSEP

Tiga pola proteksi panggilan sinkron

Pola	Fungsi	Pertanyaan yang Dijawab
Timeout	Memberi batas waktu tunggu eksplisit pada setiap panggilan keluar	Berapa lama pemanggil bersedia menunggu?
Retry + backoff	Mengulang panggilan yang gagal karena gangguan sesaat, dengan jeda yang makin panjang dan jumlah percobaan terbatas	Panggilan mana yang aman untuk diulang, dan berapa kali?
Circuit breaker	Menghentikan sementara panggilan ke service yang terbukti gagal, lalu mencoba kembali secara bertahap	Kapan berhenti mencoba agar service tujuan sempat kembali aktif?

CATATAN

Ketiga pola saling melengkapi. *Timeout* tanpa *circuit breaker* tetap membuang dua detik untuk setiap *request* ke service yang sudah jelas tidak menjawab. *Retry* tanpa *timeout* dapat menunggu selamanya pada percobaan pertama.

PETA CONCERN MICROSERVICES

Resiliency di kode dan di platform

Di kode: failsafe-go di transfer (Hari 2)

- *Timeout* membatasi waktu tunggu setiap panggilan ke account
- *Retry* mengulang panggilan yang terbukti belum diproses
- *Circuit breaker* menghentikan sementara panggilan ke account yang terus gagal
- Keputusan diambil per panggilan, berdasarkan arti setiap *error* bagi proses bisnis

Di platform: Kubernetes (Hari 4)

- *Readiness probe* gagal: *pod* dikeluarkan dari Service dan tidak menerima *request* baru
- *Liveness probe* gagal: kubelet me-*restart container* yang macet
- *Rolling update* mengganti *pod* secara bertahap. *Pod* lama tetap melayani sampai *pod* baru siap
- Keputusan diambil per *pod*, tanpa mengetahui isi *request*

CATATAN

Keduanya saling melengkapi. Selama account di-*restart* oleh Kubernetes, transfer tetap membutuhkan *timeout* dan *circuit breaker* agar tidak ikut lambat. Pada platform Spring Cloud, *circuit breaker* dipasang sebagai *library* (Resilience4j). Pada platform Kubernetes, *restart* dan *rollout* ditangani platform, sedangkan *policy* per panggilan tetap ditulis di kode karena hanya kode yang tahu panggilan mana yang aman diulang.

KEPUTUSAN DESAIN

failsafe-go untuk *timeout*, *retry*, dan *circuit breaker*

Kebutuhan

- Ketiga pola untuk panggilan transfer ke account
- Urutan *policy* dapat diatur, karena urutan menentukan perilaku
- Parameter yang sama berlaku untuk HTTP (Lab 3) dan gRPC (Lab 4)

Opsi yang dipertimbangkan

- Menulis sendiri *retry loop* dan *state machine circuit breaker* beserta *test*-nya
- sony/gobreaker: hanya *circuit breaker*, *retry* dan *timeout* ditulis terpisah
- failsafe-go

Pilihan di studi kasus

- failsafe-go: ketiga *policy* dalam satu library, disusun berurutan dalam satu pemanggilan
- Adaptor `failsafehttp` (`http.RoundTripper`) dan `failsafegrpc` (*unary client interceptor*), sehingga *policy* Lab 3 digunakan kembali di Lab 4
- Klasifikasi kegagalan ditulis sebagai fungsi biasa yang dapat diuji

Konsekuensi dan *trade-off*

- Dependensi tambahan dengan API *generic* yang perlu dipelajari tim
- *Policy* berada di kode setiap service. Service dalam bahasa lain membutuhkan library lain dengan perilaku yang diselaraskan secara manual
- Parameter disimpan di kode, sehingga perubahan nilainya membutuhkan *build* dan rilis ulang

POLA 1

Timeout pada setiap panggilan keluar

- ***Timeout* per percobaan.** Batas waktu untuk satu panggilan HTTP atau gRPC. Pada studi kasus nilainya 2 detik.
- ***Timeout* keseluruhan.** Batas waktu untuk seluruh panggilan beserta semua *retry*-nya. Nilai ini menentukan waktu tunggu yang dirasakan channel.
- Nilai *timeout* diambil dari *latency* normal service tujuan (misalnya p99) ditambah margin, lalu dicocokkan dengan batas waktu yang dijanjikan ke channel.
- Di Go, *timeout* dibawa oleh `context.Context`. *Deadline* dari pemanggil ikut membatasi semua panggilan turunannya.

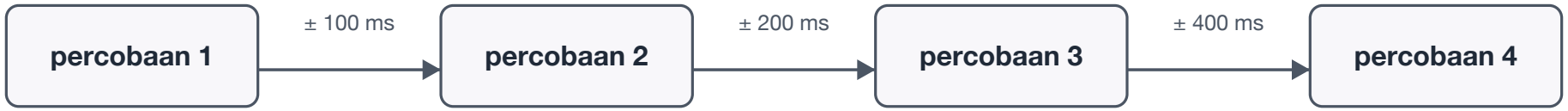
```go
// Satu percobaan dibatasi 2 detik
ctx, cancel := context.WithTimeout(
    r.Context(), 2*time.Second)
defer cancel()

req, _ := http.NewRequestWithContext(
    ctx, http.MethodPost, url, body)
resp, err := client.Do(req)
if errors.Is(err, context.DeadlineExceeded) {
    // jawaban tidak datang tepat waktu;
    // status di account TIDAK diketahui
}
```

Ilustrasi, tidak diambil dari repositori studi kasus.

POLA 2

Retry dengan *backoff* dan *jitter*



Backoff

Jeda antarpercobaan berlipat dua setiap kali, mulai 100 ms dengan batas atas 1 detik. Service tujuan yang sedang kewalahan mendapat waktu untuk kembali aktif.

Jitter

Jeda diacak ±25%. Tanpa *jitter*, ratusan transfer yang gagal bersamaan akan mengulang pada milidetik yang sama dan membebani account sekaligus.

PERHATIAN

Jumlah percobaan wajib dibatasi (studi kasus: maksimal 3 *retry*). *Retry* tanpa batas mengubah gangguan sesaat menjadi beban berlipat bagi service yang sedang bermasalah.

POLA 2

Retry hanya aman bila operasinya *idempotent*

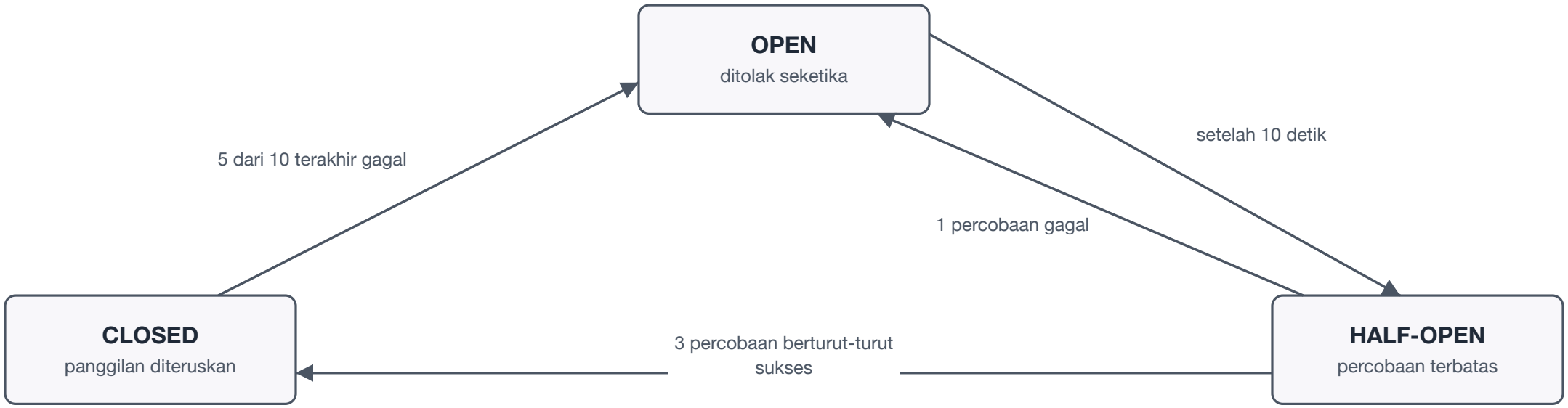
Kondisi yang Diterima Transfer	Apakah Debit Sudah Diterapkan?	*Retry* di Lab 3
Koneksi ditolak saat *dial*	Belum, *request* tidak pernah terkirim	Ya
HTTP 503 dari account	Belum, account menolak sebelum memproses	Ya
Timeout setelah *request* terkirim	Tidak diketahui, bisa saja sudah	**Tidak**
HTTP 500 atau jawaban lain yang tidak dikenal	Tidak diketahui	**Tidak**
HTTP 404 atau 422 (rekening tidak ada, saldo kurang)	Ditolak oleh aturan bisnis	Tidak, hasilnya akan sama

PERHATIAN

Debit di account belum *idempotent*, sehingga *request* yang dikirim dua kali akan mendebit dua kali. Selama kondisi itu berlaku, *timeout* tidak boleh di-*retry*. Lab 5 menambahkan *reference* unik pada setiap mutasi, dan sejak itu *timeout* ikut di-*retry*.

POLA 3

State pada *circuit breaker*



- Selama OPEN, transfer tidak menunggu *timeout* sama sekali. Panggilan langsung gagal dengan `circuitbreaker.ErrOpen`, dan account mendapat jeda untuk kembali aktif.
- Setiap perubahan *state* dicatat di log, sehingga transisi CLOSED, OPEN, dan HALF-OPEN dapat diamati langsung di Lab 3.

POLA 3

Kegagalan yang dihitung oleh *circuit breaker*

Dihitung sebagai kegagalan

- *Error* transport: koneksi ditolak, koneksi terputus
- *Timeout* per percobaan
- HTTP 5xx, termasuk 503 dari *fault injection*

Dihitung sebagai sukses

- HTTP 2xx
- HTTP 4xx: rekening tidak ditemukan (404), saldo tidak cukup (422)

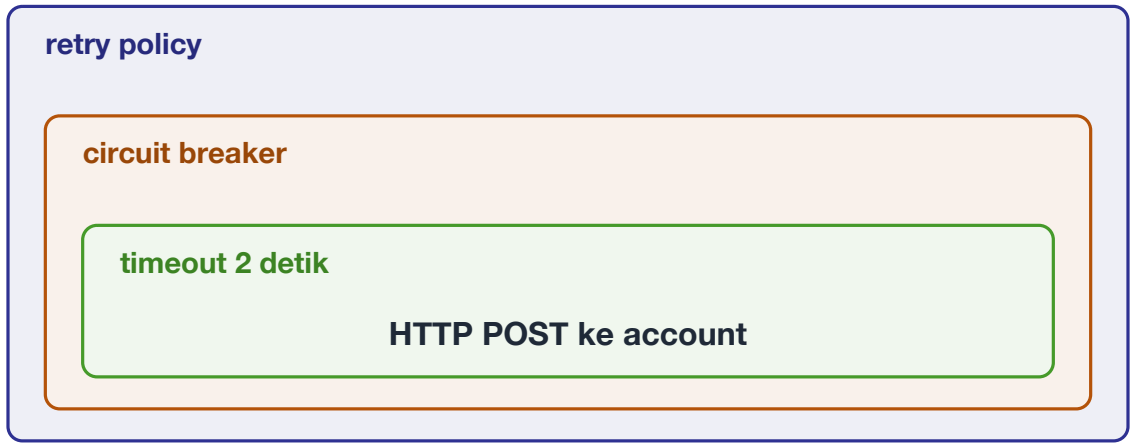
CATATAN

HTTP 4xx berarti account sehat dan menjawab sesuai aturan bisnis. Bila 4xx ikut dihitung, nasabah yang berulang kali mencoba transfer dengan saldo kurang dapat membuka *circuit breaker* dan menghentikan transfer semua nasabah lain.

TIPS

Cukup satu *circuit breaker* untuk seluruh account service karena debit dan credit berada pada service yang sama. Bila account mati, keduanya ikut gagal.

Pengaruh urutan *policy* terhadap perilaku



- ***Timeout* di dalam.** Setiap percobaan mendapat jatah 2 detik sendiri, tidak berbagi dengan percobaan sebelumnya.
- ***Circuit breaker* di tengah.** Setiap percobaan, termasuk yang berakhir *timeout*, dicatat oleh *circuit breaker*.
- ***Retry* di luar.** Bila *circuit breaker* sudah OPEN, *retry* dihentikan seketika karena percobaan berikutnya pasti ditolak.

PERHATIAN

Bila *timeout* diletakkan di luar *retry*, batas 2 detik berlaku untuk seluruh percobaan sekaligus, dan *circuit breaker* tidak pernah melihat *timeout* sebagai kegagalan.

Round tripper dengan tiga *policy*

```
breaker := circuitbreaker.NewBuilder[*http.Response]().
    HandleIf(countsAsFailure).
    WithFailureThresholdRatio(5, 10).
    WithDelay(10 * time.Second).
    WithSuccessThreshold(3).
    OnStateChanged(logStateChange).
    Build()

retry := retrypolicy.NewBuilder[*http.Response]().
    HandleIf(notProcessed).
    AbortOnErrors(circuitbreaker.ErrOpen).
    WithMaxRetries(3).
    WithBackoff(100*time.Millisecond, time.Second).
    WithJitterFactor(0.25).
    Build()

rt := failsafehttp.NewRoundTripper(nil,
    retry, breaker, timeout.New[*http.Response](2*time.Second))
client := &http.Client{Transport: rt}
```

Sumber: `transfer/internal/accountclient/client.go` · *tag* `lab-03`

- `failsafehttp` membungkus `http.RoundTripper`, sehingga kode pemanggil tetap menggunakan `http.Client` biasa
- Urutan argumen mengikuti urutan *wrapping*: *retry*, *circuit breaker*, *timeout*
- Parameter disimpan di kode dalam struct `Settings`. Perubahan nilainya melalui *review* dan *test* seperti perubahan kode lain

Klasifikasi kegagalan ditulis sebagai fungsi

```go
// Retry hanya bila request terbukti belum diproses.
// Timeout TIDAK di-retry: debit belum idempotent,
// account bisa saja sudah mendebit.
func notProcessed(resp *http.Response, err error) bool {
    if err != nil {
        var opErr *net.OpError
        return errors.As(err, &opErr) && opErr.Op == "dial"
    }
    return resp.StatusCode == http.StatusServiceUnavailable
}

// 4xx = account sehat yang menolak secara bisnis.
func countsAsFailure(resp *http.Response, err error) bool {
    return err != nil || resp.StatusCode >= 500
}
```

Sumber: `transfer/internal/accountclient/client.go` · *tag* `lab-03`

- `notProcessed` menjawab pertanyaan *retry*: apakah *request* terbukti belum diproses?
- `countsAsFailure` menjawab pertanyaan *circuit breaker*: apakah account sedang bermasalah?
- Kedua fungsi diuji terpisah dengan `httptest`, tanpa account sungguhan

SIMULASI

Fault injection pada account service

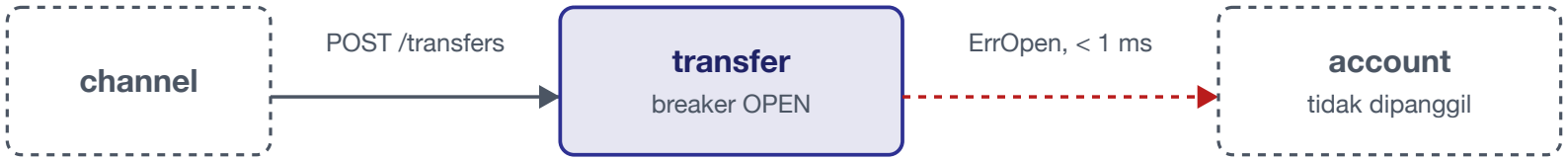
Perintah	Efek pada Account	Yang Diamati di Transfer
`make fault-slow`	Setiap *request* ke `/accounts*` ditunda 3 detik	*Timeout* 2 detik, tanpa *retry*, *circuit breaker* terbuka setelah 5 kegagalan
`make fault-error RATE=0.2`	Seperlima *request* dijawab 503 tanpa diproses	*Retry* dengan *backoff*, sebagian besar transfer tetap berhasil
`make fault-error RATE=0.5`	Separuh *request* dijawab 503 tanpa diproses	*Circuit breaker* terbuka setelah dua atau tiga transfer, karena setiap *retry* ikut dihitung
`make fault-error RATE=1`	Semua *request* dijawab 503	*Retry* habis, *circuit breaker* terbuka, transfer langsung gagal
`make fault-clear`	Account kembali normal	HALF-OPEN setelah 10 detik, lalu CLOSED setelah 3 sukses
`make breaker`	Tidak ada	`{"account":"closed|open|half-open"}`

CATATAN

Fault injection hanya berlaku untuk `/accounts*`. *Endpoint* `/healthz`, `/readyz`, dan `/admin/fault` tetap cepat, sehingga kondisi account tetap dapat diperiksa selama simulasi.

PERILAKU

Transfer ketika *circuit breaker* terbuka



- Transfer dicatat dengan status `FAILED` beserta alasannya.
- Channel menerima HTTP 503 seketika, tidak menunggu 2 detik. Debit belum diterapkan, jadi *request* aman diulang nanti.
- `GET /debug/circuit-breaker` pada transfer menampilkan *state* saat itu.

PERHATIAN

Circuit breaker dapat terbuka di antara debit dan credit. Debit sudah diterapkan, credit ditolak, dan uang tertahan di luar kedua rekening. Channel menerima 502, tidak 503, karena *request* ini tidak boleh diulang. Celah ini sudah ada sejak Lab 2 dan baru ditutup oleh saga di Lab 5.

PERILAKU

Kode HTTP `POST /transfers` di Lab 3 dan Lab 4

Hasil Transfer	HTTP	Arti bagi Channel
Transfer selesai (`COMPLETED`)	`201`	Selesai
Debit ditolak: rekening sumber tidak ada atau saldo kurang	`422`	Tidak ada mutasi. Mengulang dengan data yang sama memberi hasil yang sama
Circuit breaker terbuka saat debit	`503`	Tidak ada mutasi, aman diulang setelah jeda
Debit gagal karena account *error* atau *timeout*	`502`	Hasil debit tidak pasti, periksa `GET /transfers/{id}`
Credit gagal setelah debit, termasuk rekening tujuan tidak ada dan *breaker* yang baru terbuka	`502`	Debit sudah diterapkan dan tidak boleh diulang. Log `ERROR` mencatat bahwa *reversal* harus dilakukan manual

CATATAN

Semua kegagalan credit setelah debit dibungkus `ErrCreditFailed` di `Service.Execute` , dan *handler* memeriksanya sebelum kasus `422` dan `503` . Skenario demo yang selalu dapat diulang: transfer ke `to_account` yang tidak terdaftar menghasilkan `502` dengan `failure_reason` `credit failed after debit: ... account not found` .

LAB 3

Resiliency pada panggilan transfer ke account

Tujuan

Transfer tetap cepat merespons ketika account lambat atau mati, dan tidak pernah mendebit dua kali akibat *retry*.

Yang dikerjakan

- *Timeout* 2 detik per percobaan di `transfer/internal/accountclient`
- *Retry* maksimal 3 kali dengan *backoff* 100 ms hingga 1 detik dan *jitter*, hanya untuk *dial error* dan 503
- *Circuit breaker* 5 dari 10, jeda 10 detik, 3 sukses untuk menutup kembali
- `GET /debug/circuit-breaker` dan log perubahan *state*
- *Unit test* dengan `httptest`: *retry* pada 503, tanpa *retry* pada *timeout*, *circuit breaker* menolak dengan cepat

Rujukan

- Panduan lab: bab **Lab 3**
- Titik awal: `git checkout lab-02`
- Hasil akhir: *checkpoint* `lab-03`
- Waktu: ± 60 menit

TIPS

Jalankan `make breaker` di terminal terpisah setiap beberapa detik selama simulasi untuk melihat transisi *state*.

LAB 3

Skenario pengamatan di akhir lab

1. `make fault-error RATE=0.2`, lalu kirim beberapa transfer. Log transfer menampilkan `retrying account call`, dan sebagian besar transfer berstatus `COMPLETED`.
2. `make fault-slow`, lalu kirim 5 transfer. Setiap transfer gagal setelah ± 2 detik tanpa *retry*, kemudian log menampilkan `circuit breaker state changed` dari `closed` ke `open`.
3. Kirim transfer berikutnya. Jawaban 503 datang seketika dan account tidak menerima *request* apa pun.
4. `make fault-clear`, tunggu 10 detik, lalu kirim 3 transfer. *State* berpindah ke `half-open`, kemudian `closed`.
5. Bandingkan saldo rekening dengan daftar transfer yang `COMPLETED`. Tidak ada debit ganda.

> **CATATAN**
>
> Hasil langkah 5 menjadi bukti bahwa keputusan tidak me-*retry timeout* sudah tepat untuk debit yang belum *idempotent*.

02

gRPC untuk Komunikasi Internal

Kontrak protobuf, *code generation*, *unary call*, *error handling*, dan *interceptor*

PERBANDINGAN

REST dan gRPC

Aspek	REST + JSON	gRPC
Kontrak	OpenAPI, ditulis terpisah dan bersifat opsional	File `.proto`, wajib, menjadi sumber kode hasil *generate*
Kontrak tidak cocok terdeteksi	Saat *runtime*	Saat kompilasi
Payload	Teks JSON	Biner protobuf
Transport	HTTP/1.1 atau HTTP/2	HTTP/2
Jenis panggilan	*Request* dan *response*	*Unary*, serta *streaming* dari server, dari client, atau dua arah
Batas waktu tunggu pemanggil	Tidak dikirim ke server	Dikirim ke server sebagai *deadline*
Error	*Status code* HTTP dan isi *response*	*Status code* gRPC dan pesan *error*
Mencoba secara manual	curl, browser, Postman	grpcurl atau Postman, dengan *reflection* atau file `.proto`
Load balancing di belakang Kubernetes Service	Client membuka beberapa koneksi, sehingga panggilan tersebar ke beberapa *pod*	Satu koneksi HTTP/2 untuk semua panggilan, sehingga panggilan menumpuk di satu *pod* tanpa *client-side load balancing*
Cocok untuk	API channel dan pihak eksternal dengan client yang beragam	Panggilan internal antarservice yang dikembangkan tim sendiri

CATATAN

Keduanya digunakan bersamaan di studi kasus: REST untuk channel, gRPC untuk panggilan transfer ke account.

KEPUTUSAN DESAIN

gRPC untuk panggilan internal

Kebutuhan

- Panggilan internal bervolume tinggi: satu *request* channel memicu beberapa panggilan ke account
- Kedua sisi dikembangkan tim internal, dan kontrak harus tetap cocok setiap kali salah satu sisi berubah
- Server perlu tahu batas waktu tunggu pemanggil
- Satu titik untuk log, autentikasi, dan *resiliency*

Opsi yang dipertimbangkan

- REST + JSON seperti Lab 2, dengan kontrak OpenAPI sebagai dokumen terpisah
- gRPC dengan kontrak protobuf
- *Messaging*: tidak cocok untuk debit, karena transfer membutuhkan hasilnya saat itu juga

Pilihan di studi kasus

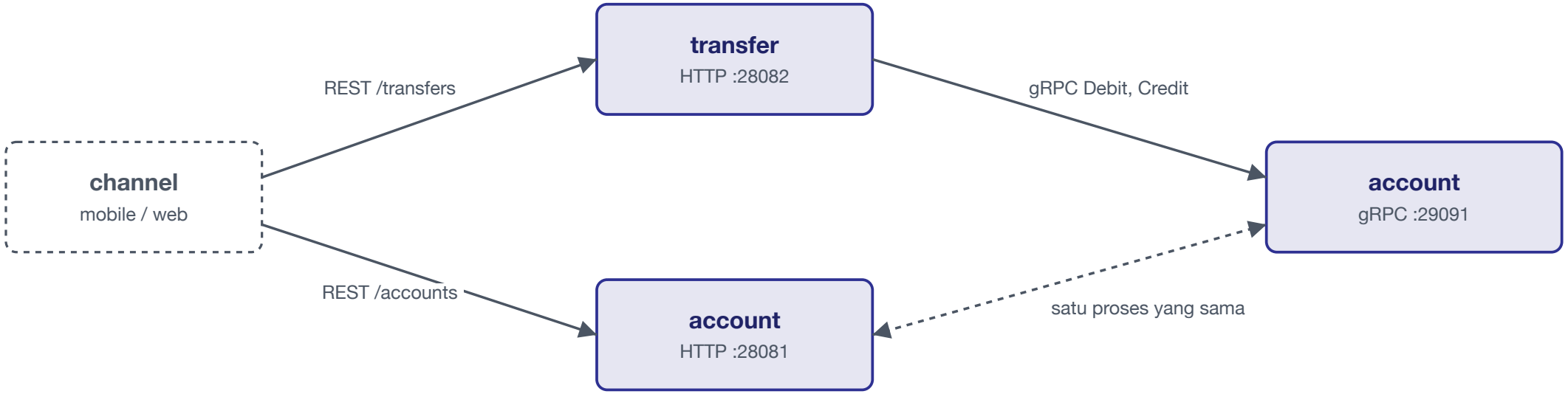
- gRPC untuk transfer ke account. API channel tetap REST + JSON karena client-nya beragam dan berada di luar kendali tim
- Kontrak protobuf dan *code generation*: ketidakcocokan tertangkap saat kompilasi
- *Deadline* ikut terkirim ke server, HTTP/2 *multiplexing*, *payload* biner
- *Interceptor* untuk log, *fault injection*, *resiliency*, dan JWT

Konsekuensi dan *trade-off*

- Tidak dapat dicoba langsung dengan curl atau browser. Perlu client gRPC seperti grpcurl, dibantu *reflection*
- Koneksi HTTP/2 persisten: Kubernetes Service membagi koneksi, tidak membagi panggilan. Pembagian per panggilan membutuhkan *client-side load balancing* atau L7 *proxy*
- Perubahan kontrak harus kompatibel, dan tim memelihara dua gaya API

STUDI KASUS

Penempatan REST dan gRPC setelah Lab 4



- Account melayani dua *port* dalam satu proses: REST untuk channel (buka rekening, cek saldo) dan gRPC untuk service internal (`GetAccount`, `Debit`, `Credit`).
- *Endpoint* REST debit dan credit di account dihapus. Mutasi saldo hanya dapat dilakukan melalui gRPC oleh service internal.

KONSEP

Komponen gRPC

Kontrak protobuf

File `.proto` mendefinisikan service, method, dan struktur data. *File* ini menjadi acuan tunggal bagi server dan client.

Code generation

`protoc` menghasilkan kode Go yang *type-safe*. Ketidakcocokan nama method dan tipe tertangkap saat kompilasi.

HTTP/2 dan *encoding* biner

Satu koneksi melayani banyak panggilan sekaligus. Ukuran data lebih kecil dan *parsing* lebih cepat dibandingkan JSON.

Interceptor

Fungsinya seperti *middleware*, tetapi untuk setiap panggilan gRPC. Log, autentikasi, *fault injection*, dan resiliency cukup ditulis sekali untuk semua method.

CATATAN

gRPC mendukung empat jenis panggilan: *unary*, *server streaming*, *client streaming*, dan *bidirectional streaming*. Studi kasus hanya menggunakan *unary* (satu *request*, satu *response*), jenis yang paling sering digunakan antarservice.

KONTRAK

Cuplikan `account/v1/account.proto`

```proto
syntax = "proto3";

package account.v1;

option go_package = ".../studi-kasus/proto/gen/account/v1;accountv1";

service AccountService {
  rpc GetAccount(GetAccountRequest) returns (GetAccountResponse);
  rpc Debit(DebitRequest) returns (DebitResponse);
  rpc Credit(CreditRequest) returns (CreditResponse);
}

// Format: digits with at most 2 decimal places
// ("150000.50"), no sign, no exponent; anything
// else is INVALID_ARGUMENT, never rounded.
message DebitRequest {
  string number = 1;
  string amount = 2;
  string reference = 3;
}
```

Sumber: `proto/account/v1/account.proto` · *tag* `lab-04`

- `package account.v1` memuat versi di nama *package*. Perubahan yang merusak kompatibilitas masuk ke `v2`
- Yang dikirim lewat jaringan hanya nomor *field* (`= 1`, `= 2`), tanpa nama *field*
- Setiap RPC memiliki *message request* dan *response* sendiri, sehingga masing-masing dapat ditambah *field* tanpa memengaruhi yang lain
- Nominal uang ditulis sebagai *string* desimal, alasannya di *slide* berikut

KONTRAK

Nominal uang tidak dikirim sebagai `double`

```
var a, b float64 = 0.1, 0.2
fmt.Println(a + b)
// 0.30000000000000004

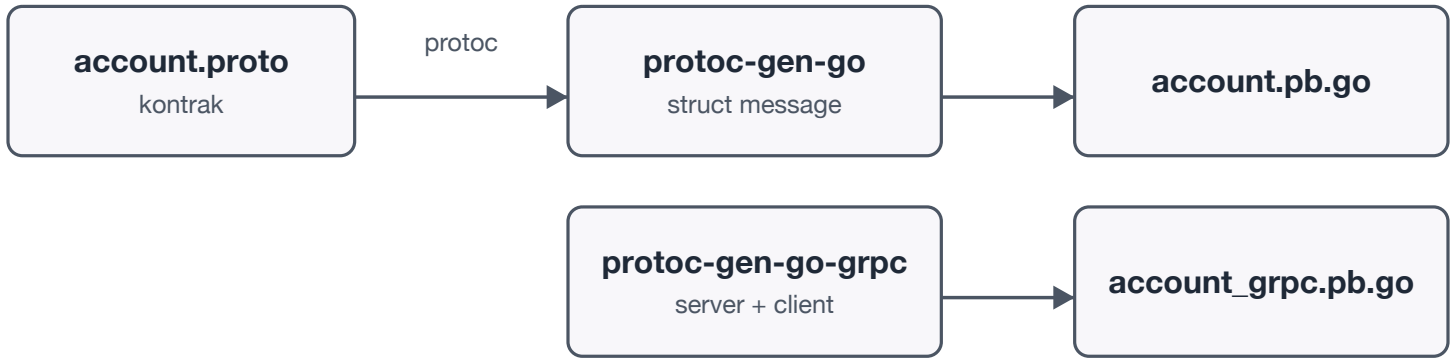
x, _ := decimal.NewFromString("0.1")
y, _ := decimal.NewFromString("0.2")
fmt.Println(x.Add(y))
// 0.3
```

Ilustrasi, tidak diambil dari repositori studi kasus.

- `double` dan `float` adalah bilangan biner berpresisi terbatas. Nilai 0,1 tidak dapat disimpan secara tepat.
- Selisih kecil tersebut menumpuk pada penjumlahan saldo dan membuat rekonsiliasi tidak cocok.
- Kontrak mengirim *string* desimal. Server dan client mengubahnya menjadi `decimal.Decimal` (shopspring/decimal), sama dengan kolom `numeric(19,2)` di database.
- Format yang diterima: angka dengan maksimal 2 desimal, tanpa tanda +/- dan tanpa eksponen, kurang dari 10^{17}. Selain itu ditolak dengan `codes.InvalidArgument` (REST: `400`).
- Nilai `100.005` ditolak, tidak dibulatkan. Tanpa pemeriksaan ini, `numeric(19,2)` membulatkannya menjadi `100.01` tanpa *error* maupun peringatan.

CODE GENERATION

Dari `.proto` menjadi kode Go



- Kontrak berada di modul tersendiri, `studi-kasus/proto`. Kode hasil *generate* ada di `proto/gen/account/v1` (*package* `accountv1`) dan ikut di-*commit*.
- `make -C proto generate` menjalankan `protoc` dengan kedua *plugin*. Pastikan `$(go env GOPATH)/bin` ada di `PATH`.
- Modul account dan transfer mengimpor modul proto dengan *directive* `replace` ke `../proto`, sehingga `go mod tidy` dan *build* Docker tetap berjalan.

SERVER

Account melayani HTTP dan gRPC sekaligus

```
grpcSrv := grpc.NewServer(grpc.ChainUnaryInterceptor(
    interceptor.Logging(log),
    faults.UnaryServerInterceptor(notAccountService),
))
accountv1.RegisterAccountServiceServer(grpcSrv,
    account.NewGRPCServer(store, log))
healthSrv := health.NewServer()
healthpb.RegisterHealthServer(grpcSrv, healthSrv)
reflection.Register(grpcSrv)

lis, err := net.Listen("tcp", cfg.GRPCAddr)
if err != nil {
    return fmt.Errorf("listen on GRPC_ADDR %s: %w",
        cfg.GRPCAddr, err)
}
go func() { errCh <- grpcSrv.Serve(lis) }()
```

Sumber: `account/main.go` · *tag* `lab-04`

- `GRPC_ADDR` wajib diisi. Bila kosong, account berhenti saat *startup* dan log *error*-nya menyebut nama variabel tersebut
- *Health service* standar gRPC (`grpc.health.v1`) tidak memeriksa database, jadi statusnya `SERVING` sejak *startup* hingga *shutdown*. *Readiness pod* tetap diperiksa melalui HTTP `/readyz`
- *Reflection* memungkinkan client generik membaca daftar method tanpa *file* `.proto`
- HTTP dan gRPC berjalan di *goroutine* terpisah dalam satu proses
- `notAccountService` bernilai `true` untuk method di luar `AccountService` (*health*, *reflection*), sehingga method tersebut tidak terkena *fault injection*. Di kode Lab 4 fungsi ini ditulis langsung sebagai argumen, dan variabel dengan nama ini baru muncul di Lab 7

Pemetaan *error* ke *status code* gRPC

Kondisi di Account	*Status Code*	Tindakan Pemanggil
Rekening tidak ditemukan	`NotFound`	Transfer gagal, tidak di-*retry*
Saldo tidak cukup	`FailedPrecondition`	Transfer gagal, tidak di-*retry*
Nominal atau nomor rekening tidak valid	`InvalidArgument`	Transfer gagal, tidak di-*retry*
Fault injection menolak sebelum memproses	`Unavailable`	Aman di-*retry*
Deadline pemanggil terlampaui saat *query* masih berjalan	`DeadlineExceeded`	Mutasi mungkin sudah diterapkan, tidak di-*retry* (hingga Lab 5)
Pemanggil membatalkan panggilan	`Canceled`	Sama dengan `DeadlineExceeded`
Error lain yang tidak terduga	`Internal`	Tidak di-*retry*, dihitung oleh *circuit breaker*

CATATAN

Status code setara dengan HTTP *status* pada REST. Kode `NotFound` dan `FailedPrecondition` adalah jawaban bisnis dari service yang sehat, sedangkan `Unavailable` dan `DeadlineExceeded` menandakan service yang bermasalah. Account mencatat `DeadlineExceeded` dan `Canceled` sebagai `WARN caller gave up before the request finished`, tidak sebagai `ERROR`.

Status code di sisi server dan client

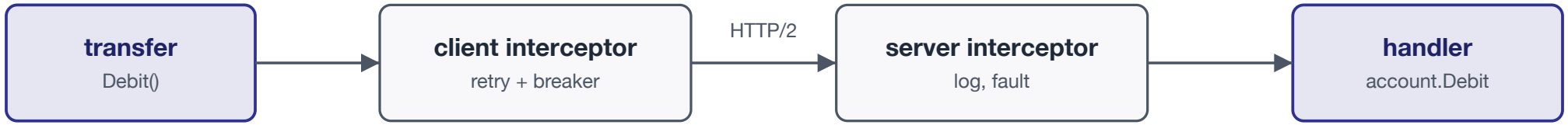
```go
// account: error domain -> status gRPC
switch {
case errors.Is(err, context.Canceled),
    errors.Is(err, context.DeadlineExceeded):
    st := status.FromContextError(err)
    s.log.Warn("caller gave up before the request finished",
        "code", st.Code().String(), "error", err)
    return st.Err()
case errors.As(err, &v): // ValidationError
    return status.Error(codes.InvalidArgument, v.describe())
case errors.Is(err, ErrNotFound):
    return status.Error(codes.NotFound, err.Error())
case errors.Is(err, ErrInsufficientFunds):
    return status.Error(codes.FailedPrecondition, err.Error())
default:
    s.log.Error("grpc request failed", "error", err)
    return status.Error(codes.Internal, "internal error")
}

// transfer: status gRPC -> error domain
switch status.Code(err) {
case codes.NotFound:
    return fmt.Errorf("%s %s: %w", kind, number,
        transfer.ErrAccountNotFound)
// ...
```

Sumber: `account/internal/account/grpc.go`, `transfer/internal/accountclient/client.go` · *tag* `lab-04`

- Server mengubah *error* domain menjadi `status.Error` dengan kode yang sesuai. *Context* yang berakhir dipetakan oleh `status.FromContextError`
- Client membaca kode dengan `status.Code(err)` lalu mengembalikan *error* domain transfer yang sama seperti versi REST
- Kode di *package* `transfer` tidak berubah. Hanya `accountclient` yang diganti

Interceptor membungkus setiap panggilan gRPC



Unary server interceptor

- Dipasang dengan `grpc.ChainUnaryInterceptor`, dijalankan berurutan
- Log setiap panggilan: method, *status code*, durasi
- *Fault injection* yang sama dengan versi REST, menjawab `codes.Unavailable`
- Lab 7 menambahkan pemeriksaan JWT di posisi yang sama

Unary client interceptor

- Dipasang saat membuat koneksi dengan `grpc.WithUnaryInterceptor`
- *Policy* failsafe-go dari Lab 3 dipindahkan ke sini
- Berlaku untuk `GetAccount`, `Debit`, dan `Credit` tanpa mengubah kode pemanggil

Bentuk *unary server interceptor*

```go
func Logging(log *slog.Logger) grpc.UnaryServerInterceptor {
    return func(ctx context.Context, req any,
        info *grpc.UnaryServerInfo,
        handler grpc.UnaryHandler) (any, error) {

        start := time.Now()
        resp, err := handler(ctx, req)
        code := status.Code(err)
        level := slog.LevelInfo
        switch code {
        case codes.OK, codes.NotFound,
            codes.FailedPrecondition, codes.InvalidArgument:
        default:
            level = slog.LevelWarn
        }
        log.Log(ctx, level, "grpc request",
            "method", info.FullMethod, "code", code.String(),
            "duration_ms", time.Since(start).Milliseconds())
        return resp, err
    }
}
```

Sumber: `account/internal/interceptor/logging.go` · *tag* `lab-04`

- `handler` adalah method service yang sesungguhnya
- Kode sebelum pemanggilan `handler` berjalan sebelum method, sedangkan kode sesudahnya dapat membaca hasil method
- `info.FullMethod` berisi nama lengkap, contoh `/account.v1.AccountService/Debit`
- Jawaban bisnis dicatat `INFO`, kode lain yang gagal dicatat `WARN`
- Pola yang sama dengan *middleware* `http.Handler` dari Hari 1

RESILIENCY

Policy Lab 3 dalam bentuk *status code* gRPC

Status Code	*Retry*	*Circuit Breaker*	Alasan
`Unavailable`	Ya	Gagal	Account menolak sebelum memproses, atau koneksi tidak terbentuk
`DeadlineExceeded`	Tidak (hingga Lab 5)	Gagal	Debit mungkin sudah diterapkan
`Internal`, `Unknown`	Tidak	Gagal	Account bermasalah, status mutasi tidak diketahui
`NotFound`, `FailedPrecondition`, `InvalidArgument`	Tidak	Sukses	Jawaban bisnis dari service yang sehat

CATATAN

Parameter *policy* tetap sama dengan Lab 3: *timeout* 2 detik per percobaan, maksimal 3 *retry* dengan *backoff* 100 ms hingga 1 detik, *circuit breaker* 5 dari 10.

TIPS

`GET /debug/circuit-breaker` dan `make fault-*` tetap berfungsi, sehingga skenario Lab 3 dapat diulang pada jalur gRPC.

PERHATIAN

`Unavailable` juga muncul bila koneksi terputus setelah *request* terkirim. Risiko debit ganda ini sengaja dibiarkan sampai Lab 5 membuat mutasi account menjadi *idempotent*.

CLIENT

Koneksi gRPC di transfer service

```go
// accountclient.New
resilience := failsafegrpc.NewUnaryClientInterceptor[any](
    retry, breaker, timeout.New[any](s.AttemptTimeout))

conn, err := grpc.NewClient(target,
    grpc.WithTransportCredentials(insecure.NewCredentials()),
    grpc.WithUnaryInterceptor(resilience),
    grpc.WithConnectParams(grpc.ConnectParams{
        Backoff: backoff.Config{BaseDelay: time.Second,
            Multiplier: 1.6, Jitter: 0.2,
            MaxDelay: 5 * time.Second},
        MinConnectTimeout: 5 * time.Second,
    }),
)

// Deadline per percobaan berasal dari policy timeout
// dan ikut terkirim ke server (header grpc-timeout).
_, err := c.api.Debit(ctx, &accountv1.DebitRequest{
    Number: number, Amount: amount.String(),
    Reference: reference})
return mapError("debit", number, err)
```

Sumber: `transfer/internal/accountclient/client.go` · *tag* `lab-04`

- `ACCOUNT_GRPC_ADDR` menggantikan `ACCOUNT_SERVICE_URL` dan wajib diisi
- `grpc.NewClient` tidak langsung membuka koneksi. Koneksi dibuat saat panggilan pertama dan dibuat ulang otomatis bila terputus, dengan jeda maksimal 5 detik
- `insecure.NewCredentials()` hanya untuk lab. Di *production*, koneksi antarservice wajib TLS

KONSEP

Deadline ikut terkirim ke server



- Pada REST, *deadline* pemanggil tidak dikirim ke server. Server baru tahu pemanggil berhenti menunggu ketika koneksi diputus, dan hanya bila kode server memeriksa `r.Context()`.
- Pada gRPC, *deadline* dari `context` pemanggil dikirim sebagai *header* `grpc-timeout`. `ctx` di sisi server ikut berakhir pada waktu yang sama.
- Account meneruskan `ctx` tersebut ke pgx, sehingga *query* yang tidak lagi ditunggu dibatalkan dan koneksi database segera dilepas.

KONTRAK

Kompatibilitas perubahan kontrak protobuf

Aman (kompatibel)

- Menambah *field* baru dengan nomor baru
- Menambah method baru pada service
- Menambah nilai baru pada `enum`
- Client lama tetap berjalan: *field* yang tidak dikenal diabaikan

Merusak kompatibilitas

- Mengubah nomor atau tipe *field* yang sudah ada
- Menggunakan kembali nomor *field* yang sudah dihapus
- Mengganti nama atau menghapus method
- Sebagai gantinya, tandai nomor lama dengan `reserved` atau buat *package* baru `account.v2`

CATATAN

Kompilasi hanya menangkap nama method atau *field* yang diganti atau dihapus, pada service yang di-*build* dari kontrak baru. Nomor *field* yang diubah atau digunakan kembali tidak menimbulkan *compile error*, tetapi merusak data antara versi lama dan versi baru yang berjalan bersamaan. Karena itu nomor lama ditandai `reserved`, dan perubahan kontrak diperiksa dengan *breaking-change tool* atau *review*.

TESTING

Unit test gRPC dengan `bufconn`

```
fake := &fakeAccountService{answer: answer}
lis := bufconn.Listen(1 << 20)
srv := grpc.NewServer()
accountv1.RegisterAccountServiceServer(srv, fake)
go func() { _ = srv.Serve(lis) }()
t.Cleanup(srv.Stop)

// Client yang sama dengan production, ditambah
// dialer ke listener di memori.
c, err := New("passthrough:///bufnet", s, log,
    grpc.WithContextDialer(
        func(ctx context.Context, _ string) (net.Conn, error) {
            return lis.DialContext(ctx)
        }))
```

Sumber: `transfer/internal/accountclient/client_test.go` · *tag* `lab-04`

- `bufconn` menyediakan *listener* di memori. Server dan client gRPC sungguhan berjalan tanpa membuka *port*
- *Fake server* di *test* dapat mengembalikan `codes.Unavailable` atau menunda jawaban
- Skenario *test* Lab 3 diulang: *retry* pada `Unavailable`, tanpa *retry* pada `DeadlineExceeded`, *circuit breaker* terbuka

LAB 4

Panggilan transfer ke account berpindah ke gRPC

Tujuan

Kontrak antara transfer dan account diperiksa saat kompilasi, dan resiliency Lab 3 tetap berlaku melalui *interceptor*.

Yang dikerjakan

- Modul `proto` dengan `account/v1/account.proto` dan kode hasil *generate*
- Server gRPC di account (`GRPC_ADDR`): pemetaan *error*, *health*, *reflection*, *interceptor* log dan *fault injection*
- *Endpoint* REST debit dan credit di account dihapus
- Client gRPC di transfer (`ACCOUNT_GRPC_ADDR`) dengan *client interceptor* failsafe-go
- *Unit test* dengan `bufconn`

Rujukan

- Panduan lab: bab **Lab 4**
- Titik awal: `git checkout lab-03`
- Hasil akhir: *checkpoint* `lab-04`
- Waktu: ± 90 menit

TIPS

Verifikasi akhir: transfer yang dikirim channel melalui REST tetap berhasil, dan `make fault-error RATE=1` kembali membuka *circuit breaker*, kali ini melalui `codes.Unavailable`.

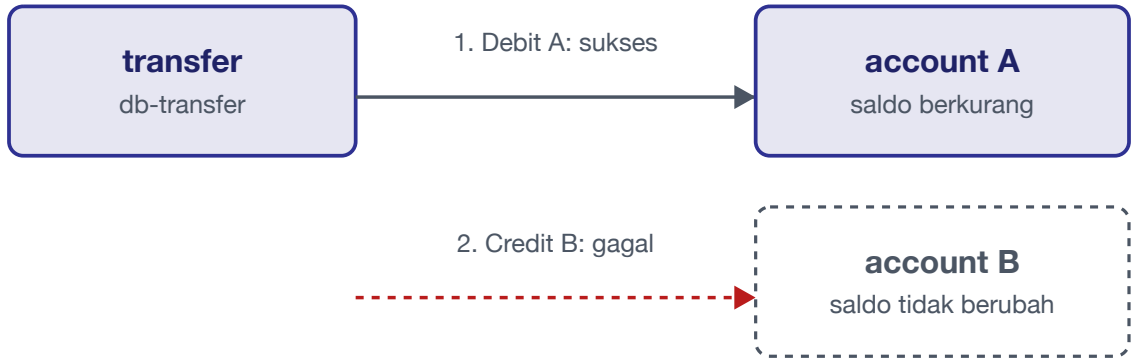
03

Konsistensi Data Terdistribusi

Alasan 2PC dihindari, saga dengan kompensasi di studi kasus, dan *idempotency key* untuk operasi finansial

MASALAH

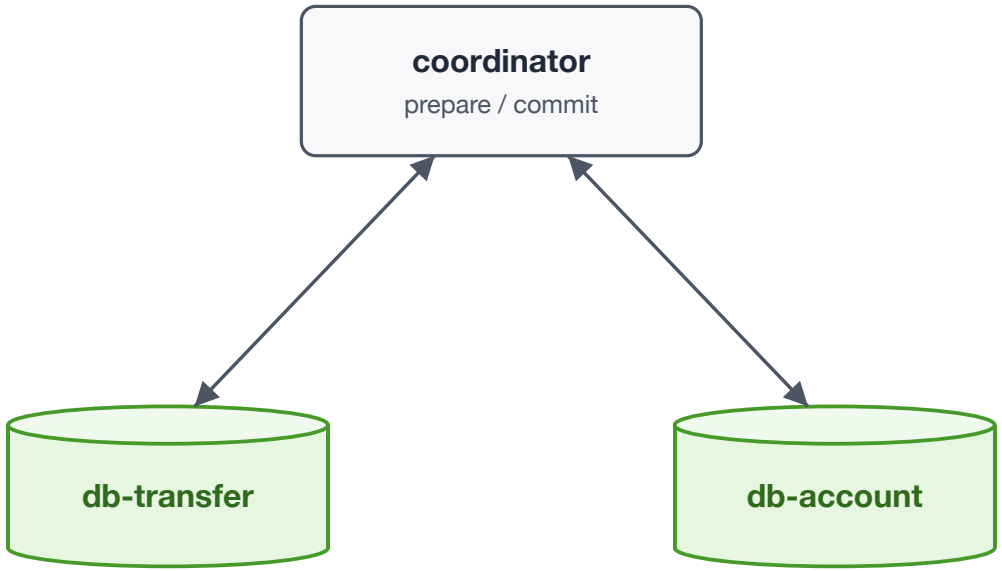
Transfer mengubah data di dua database



- Debit dan credit berjalan sebagai dua transaksi database terpisah di dua service yang berbeda. Tidak ada `BEGIN ... COMMIT` yang mencakup keduanya.
- Bila credit gagal setelah debit berhasil, saldo rekening A sudah berkurang tetapi rekening B tidak bertambah. Transfer di akhir Lab 2 hanya mencatat log untuk koreksi manual.
- Demo: transfer ke nomor rekening tujuan yang tidak ada. Debit berhasil, credit dijawab `NotFound`, channel menerima `502`, dan saldo sumber tidak kembali.

KONSEP

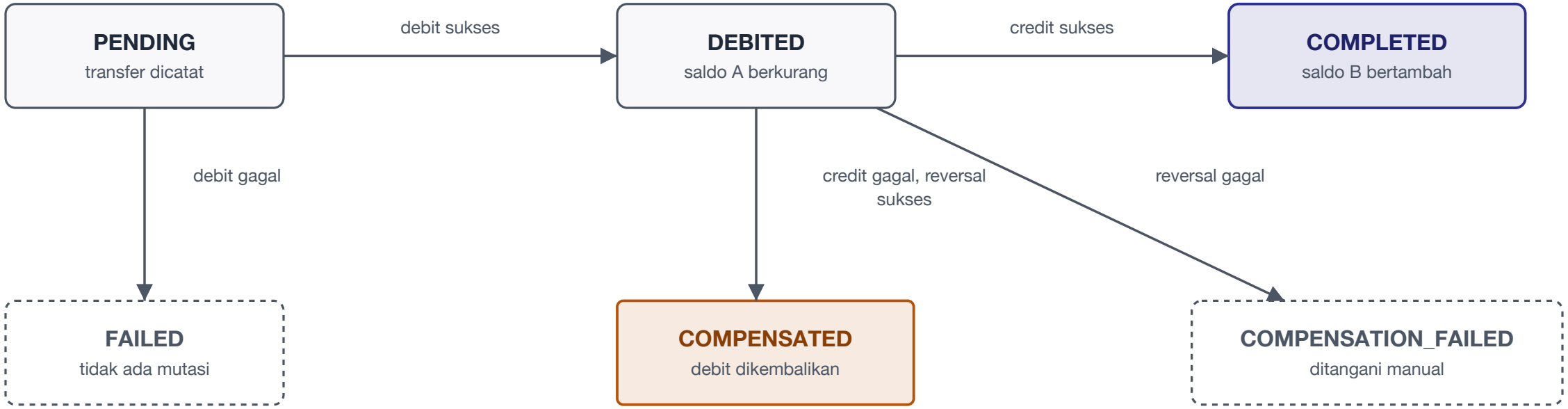
Alasan *distributed transaction* (2PC) dihindari



- ***Tight coupling*.** Semua database yang terlibat harus mendukung protokol yang sama (XA) dan aktif bersamaan.
- ***Availability* turun.** Database yang lambat menahan *lock* di semua database hingga *coordinator* memutuskan.
- ***Coordinator* menjadi *single point of failure*.** Bila *coordinator* mati setelah *prepare*, database yang terlibat tertahan menunggu.
- **Dukungan *tools* minim.** Kafka, sebagian besar *managed database* di *cloud*, dan API HTTP/gRPC tidak ikut dalam 2PC.

KONSEP

Saga sebagai rangkaian transaksi lokal dengan kompensasi



- Setiap langkah adalah transaksi lokal di satu service yang langsung di-*commit*. Bila langkah berikutnya gagal, langkah yang sudah selesai dibatalkan dengan transaksi kompensasi, yaitu credit kembali ke rekening sumber (*reversal*).
- `COMPENSATION_FAILED` dicatat sebagai log *error* dan ditangani manual oleh petugas operasional. Status ini tidak diperbaiki otomatis sebelum diperiksa.

KONSEP

Orchestration dan *choreography*

Orchestration

- Satu service (*orchestrator*) menyimpan *state* saga dan memanggil setiap langkah
- Alur mudah dibaca di satu tempat dan mudah dilacak
- *Orchestrator* mengetahui semua service yang terlibat dalam saga
- **Digunakan di studi kasus:** transfer sebagai *orchestrator* untuk debit, credit, dan *reversal*

Choreography

- Tidak ada pengendali pusat. Setiap service bereaksi terhadap *event* dari service lain
- *Coupling* antarservice lebih longgar, cocok untuk alur pendek dengan sedikit service
- Alur hanya terlihat dengan menggabungkan log semua service, dan makin sulit dilacak ketika jumlah langkah bertambah
- **Di studi kasus:** tidak ada saga *choreography*. Notification (Hari 3) hanya bereaksi terhadap *event* transfer, dengan mekanisme seperti *choreography* tetapi tanpa kompensasi

TIPS

Transfer dana membutuhkan status yang jelas setiap saat, misalnya untuk menjawab keluhan nasabah. Karena itu studi kasus memilih *orchestration*: status setiap transfer dapat dibaca dari satu tabel.

Kode HTTP `POST /transfers` sejak Lab 5

Hasil Saga	HTTP	Arti bagi Channel
`COMPLETED`	`201`	Selesai
Rekening tidak ada atau saldo kurang, termasuk `COMPENSATED` karena rekening tujuan tidak ada	`422`	Tidak ada uang yang berpindah. Mengirim ulang data yang sama tetap menghasilkan `422`
Circuit breaker terbuka saat debit (`FAILED`), atau saat credit lalu debit dikembalikan (`COMPENSATED`)	`503`	Tidak ada uang yang berpindah, aman diulang setelah jeda
Dependency gagal (`FAILED` atau `COMPENSATED`)	`502`	Periksa `GET /transfers/{id}`
`COMPENSATION_FAILED`	`500`	Dana tertahan di sisi transfer, ditangani petugas

CATATAN

Berbeda dengan Lab 3 dan Lab 4, *breaker* yang terbuka di antara debit dan credit kini menghasilkan `503` karena debit sudah dikembalikan dan tidak ada uang yang berpindah.

KONSEP

Idempotency key untuk operasi finansial

Masalah

- Transfer mengirim debit, account memproses, jawabannya hilang karena *timeout*
- Tanpa *idempotency*, *retry* akan mendebit dua kali
- Channel juga dapat mengirim ulang `POST /transfers` karena alasan yang sama

Solusi di studi kasus

- **Account (Lab 5).** *Unique constraint* pada `(account_number, type, reference)`. *Reference* yang sama tidak diterapkan ulang, account cukup mengembalikan kondisi rekening saat ini.
- **Transfer (Lab 6).** *Header* `Idempotency-Key` pada `POST /transfers`. *Key* yang sama mengembalikan transfer asli dengan kode HTTP yang sama seperti *response* pertama, sedangkan *key* yang sama dengan isi berbeda ditolak (422).

SAGA

Mutasi account dibuat *idempotent*

```
create unique index if not exists
  mutation_idempotency
  on mutation (account_number, type, reference);

insert into mutation (...) values (...)
on conflict (account_number, type, reference)
do nothing;
```

Sumber: `account/internal/account/schema.sql`, `account/internal/account/store.go` · *tag* `lab-05`

- Bila tidak ada baris yang masuk, berarti mutasi dengan *reference* tersebut sudah pernah dicatat. Account lalu membaca saldo dalam transaksi yang sama, menjawab sukses tanpa mutasi baru, dan mencatat log `mutation already applied, not repeated`
- *Reference* debit = `transfer_id`, *reference reversal* = `transfer_id:reversal`

Dampak ke klien gRPC di transfer

Di Lab 3 dan Lab 4, *retry* hanya dilakukan untuk `Unavailable`. Setelah mutasi dibuat *idempotent* di Lab 5, `DeadlineExceeded` juga boleh di-*retry*.

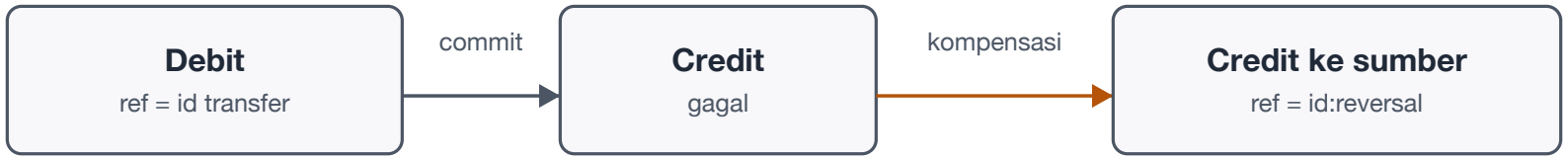
Debit yang ternyata sudah diproses sebelum *timeout* tidak akan dijalankan dua kali.

PERHATIAN

Lab 5 (Hari 3) dimulai dengan `make infra-reset` lalu `make infra-up`. Database dari Lab 1–4 dapat berisi *reference* ganda, sehingga *unique index* gagal dibuat dan account berhenti saat *startup* dengan *error* `apply schema`.

SAGA

Kompensasi berupa transaksi baru



- Kompensasi berbeda dengan *rollback*. *Rollback* menghapus jejak transaksi seolah tidak pernah terjadi, sedangkan kompensasi membalik efek bisnis (*semantic undo*) dengan mutasi credit baru ber-*reference* `<transfer_id>:reversal`. Debit asli dan *reversal* tetap tercatat untuk audit dan rekonsiliasi
- Di antara debit dan *reversal*, saldo rekening sumber sempat berkurang dan dapat terbaca oleh nasabah. Saga menjamin *eventual consistency* (data akhirnya konsisten), tetapi tidak menjamin isolasi
- Credit yang `DeadlineExceeded` diulang terlebih dahulu dengan *reference* yang sama. Bila semua percobaan berakhir *timeout*, hasilnya tidak pasti. Lab tetap menjalankan kompensasi, sedangkan sistem *production* menambahkan *reconciliation job*
- Kompensasi juga bisa gagal atau *timeout*, jadi kompensasi wajib aman untuk diulang. Kompensasi yang gagal berakhir di `COMPENSATION_FAILED`

PERHATIAN

Setiap perubahan status saga dan *event*-nya ditulis dalam satu transaksi database (*transactional outbox*). Pola ini dibahas dan dipraktikkan di Hari 3.

Reversal dengan *policy* sendiri tanpa *circuit breaker*

```go
// transfer.Service
func (s *Service) compensate(ctx context.Context,
    t Transfer, cause error) (Transfer, error) {
    reason := fmt.Sprintf("credit failed: %v", cause)
    err := s.accounts.Reverse(ctx, t.FromAccount,
        t.Amount, t.ID+ReversalSuffix)
    if err != nil {
        reason = fmt.Sprintf("%s; reversal failed: %v",
            reason, err)
        return s.finish(ctx, t, Debited, CompensationFailed,
            reason, "", fmt.Errorf("%w: %w",
                ErrReversalFailed, errors.Join(cause, err)))
    }
    return s.finish(ctx, t, Debited, Compensated,
        reason, EventCompensated, cause)
}

// accountclient.New: tanpa breaker, dibatasi durasi
reversalRetry := retrypolicy.NewBuilder[any]().
    HandleIf(countsAsFailure).
    WithMaxRetries(-1). // dibatasi durasi, bukan jumlah
    WithMaxDuration(s.ReversalMaxDuration). // 30 s
    WithBackoff(s.ReversalBackoffMin, s.ReversalBackoffMax).
    WithJitterFactor(jitterFactor).
    Build()
```

Sumber: `transfer/internal/transfer/service.go`, `transfer/internal/accountclient/client.go` · *tag* `lab-05`

- Credit biasanya gagal karena account mati atau *breaker* baru terbuka. Bila *reversal* lewat *breaker* yang sama, *reversal* langsung ditolak dan transfer berakhir `COMPENSATION_FAILED`
- *Reversal* berjalan di koneksi gRPC kedua dengan *retry* dan *backoff* 200 ms–5 detik, paling lama 30 detik
- Setiap percobaan membawa *reference* yang sama, sehingga saldo tidak bertambah dua kali
- Log `retrying reversal` menandai setiap *retry*

Saga dan *graceful shutdown*

Durasi terburuk satu saga

Langkah	Durasi
Debit: 4 percobaan × 2 detik + 3 *backoff* × 1,25 detik	11,75 detik
Credit, dengan perhitungan yang sama	11,75 detik
Reversal: paling lama 30 detik + 1 percobaan terakhir	32 detik
Total	**55,5 detik**

Akibatnya

- `shutdownGrace` transfer = 60 detik. `TestShutdownGrace` gagal bila *policy* bertambah panjang melewati angka ini
- Platform harus memberi waktu yang cukup sebelum `SIGKILL`, melalui `terminationGracePeriodSeconds` di Kubernetes (Hari 4) dan `stop_grace_period` di Docker Compose

PERHATIAN

Masih ada satu celah. Bila proses mati paksa (`SIGKILL`, *crash*) di antara dua langkah, baris `PENDING` atau `DEBITED` tertinggal tanpa ada yang melanjutkan. Sistem *production* menambahkan *recovery job* yang melanjutkan atau mengompensasi baris tersebut.

RINGKASAN

Ringkasan pola Hari 2

Pola	Masalah yang Diatasi	Layak Digunakan Ketika
Timeout	Pemanggil menunggu tanpa batas	Selalu, pada setiap panggilan jaringan
Retry + *backoff*	Gangguan sesaat menggagalkan *request*	Operasinya *idempotent* atau terbukti belum diproses
Circuit breaker	Service yang mati terus dibebani dan pemanggil ikut lambat	Panggilan sinkron ke service yang dapat gagal
gRPC	Kontrak internal hanya diperiksa saat program berjalan	Panggilan internal bervolume tinggi antartim yang sama-sama mengendalikan kontrak
Saga (*orchestration*)	Transaksi lintas database tanpa 2PC	Satu proses bisnis mengubah data di lebih dari satu service dan butuh status yang jelas
Kompensasi	Langkah yang sudah di-*commit* harus dibatalkan	Ada langkah saga yang bisa gagal setelah langkah lain berhasil
Idempotency key	*Retry* dan pengiriman ulang menggandakan mutasi	Setiap operasi yang mengubah saldo

PETA CONCERN MICROSERVICES

Posisi Hari 2 pada peta *concern*

Concern	Yang Dibangun di Hari 2 (di Kode)	Dilanjutkan di
Komunikasi sinkron dan asinkron	REST untuk channel, gRPC untuk transfer ke account (Lab 4)	*Messaging* Kafka untuk notification (Hari 3), Service DNS di Kubernetes (Hari 4)
Resiliency	*Timeout*, *retry*, dan *circuit breaker* dengan failsafe-go (Lab 3–4)	Metrik *circuit breaker* di Grafana (Lab 8). *Probe*, *restart*, dan *rolling update* di Kubernetes (Lab 9)
Konsistensi data	Desain saga transfer: status, kompensasi, kode HTTP, mutasi *idempotent*, *idempotency key*	Implementasi saga, *transactional outbox*, dan *idempotent consumer* (Lab 5–6)
Security dan *observability*	*Interceptor* gRPC sebagai tempat pemeriksaan bersama untuk semua method	Validasi JWT (Lab 7), *span* untuk setiap percobaan panggilan (Lab 8)

CATATAN

Hari 2 hanya mengisi kolom “di kode” pada peta. Bagian platform dibahas di Hari 4: Kubernetes me-*restart* account yang macet, sedangkan failsafe-go di transfer tetap melindungi pemanggil selama *restart* berlangsung.

BERIKUTNYA

Materi Hari 3

- **Kafka untuk *messaging*.** *Topic*, *partition*, *consumer group*, dan *at-least-once delivery*.
- **Lab 5.** Desain saga dari hari ini diimplementasikan di transfer, *event* dikirim melalui *transactional outbox*, notification sebagai *consumer*.
- **Lab 6.** *Idempotent consumer*: *replay event* tanpa notifikasi ganda, `Idempotency-Key` pada transfer.
- **Lab 7, *security*.** OAuth2 *client credentials* dan validasi JWT pada REST *middleware* dan gRPC *interceptor*.
- ***Observability*.** Log, metrik, dan *trace* lintas service sebagai persiapan Lab 8 di awal Hari 4.

> **TIPS**
>
> Pastikan *checkpoint* `lab-04` berjalan di laptop masing-masing sebelum Hari 3 dimulai: `make test` lulus dan transfer melalui gRPC berhasil.